Transcript Intelligence — AegisCloud Meeting Analytics

Dataset: 100 customer meetings · support escalations, external account/renewal calls, internal syncs
Products: Detect · Protect · Comply · Identity
DB: `meeting_analytics` schema @ `localhost:5434`

Vikrant Potnis | vikrant.potnis@gmail.com | Prepared for Aziro (client Rubrik) | May 11, 2026

Design Decisions — Theme Classification Pipeline

Three approaches were evaluated. Each iteration fixed the failure mode of the previous one.

Approach 1 — Rule-Based Keyword Taxonomy

How it works: A hand-coded taxonomy maps keywords and phrases to a fixed set of 8 themes using regex/LIKE matching on transcript text and summaries.

Why it fails: Not scalable and error-prone. Every theme must be anticipated upfront — themes that emerge from the data are invisible. `pipeline failure`, `ingestion pipeline down`, and `detect outage` all mean the same thing; the rules treat them as different. Each new product, feature, or shift in customer language requires manual rule updates. It breaks silently — misclassified meetings produce no error, just wrong output.

Approach 2 — TF-IDF + KMeans

How it works: Meetings are represented as TF-IDF vectors. KMeans groups them into K clusters. Cluster names are generated by concatenating the top-K highest-weight terms from each cluster centroid (e.g. `"renewal / competitive / pricing / outage"`).

Why it fails:

- **K must be specified upfront.** There is no principled way to know whether the correct number of themes is 8, 12, or 26. The wrong K forces unrelated topics

together or splits natural groupings.

- **Bag-of-words misses semantics.** TF-IDF treats `outage remediation` and `post-mortem` as unrelated because they share no tokens. Embeddings capture that they mean the same thing.
- **Short phrases kill TF-IDF signal.** The input is `topics[]` — short 2-5 word phrases. TF-IDF on short text produces sparse, noisy vectors. Silhouette score on this dataset: **0.04** (near-random).
- **Centroid-term labels don't work in embedding space.** Top centroid terms are readable in TF-IDF space because each dimension corresponds to a word. In embedding space, dimensions are abstract latent features — there are no top terms to read off. An LLM is required to name the cluster.

Approach 3 — Embeddings + HDBSCAN + LLM Labeling

Embed phrases, not meetings. Each meeting's `topics[]` contains atomic business concepts (e.g. `api rate limiting`, `hipaa compliance`). Each unique phrase is embedded independently. A meeting about HIPAA *and* API failures maps to two distinct clusters rather than one blended vector that lands in neither.

UMAP reduces dimensions before clustering (768 → 10 dims). HDBSCAN degrades in high-dimensional space. A separate 2-dim UMAP is computed for visualisation only.

HDBSCAN finds K from the data. No fixed K required. Found 26 clusters naturally. 22 noise phrases (6.4%) were reassigned to the nearest centroid.

LLM labels the clusters — it does not form them. The 20 phrases nearest to each cluster centroid are sent to `llama3.1:8b`, which returns a theme title, audience, and rationale. The grouping is decided by HDBSCAN; the LLM only names what is already there. This is why a small local model produces clean labels.

Database Schema Decision — Two-Layer Table Design

Base tables (6) — loaded from raw JSON:
`meetings`, `meeting_participants`, `meeting_summaries`, `key_moments`, `action_items`, `transcript_lines`

Loaded first, independent of clustering. Represent what the transcripts contain — sentiment, signals, participants, action items.

Semantic tables (3) — loaded from clustering outputs:
`semantic_clusters`, `semantic_phrases`, `semantic_meeting_themes`

Loaded second. All insight queries join base and semantic tables within the same `meeting_analytics` schema.

Why separate layers: Base tables can be reloaded from raw JSON at any time without touching clustering. Semantic tables can be rebuilt from pre-computed CSVs in seconds, or regenerated by re-running the full pipeline. `semantic_meeting_themes` stores one row per (meeting, cluster) — not a `meeting_ids[]` array per cluster — because `is_primary` is a junction attribute and `sentiment` / `call_type` are denormalized here for join-free aggregation.

Setup. Connect to PostgreSQL and define three helpers used throughout: `q()` runs any SQL query and returns a DataFrame, `save()` exports a chart to `outputs_notebook/`, and `label_bars()` adds value labels to bar charts.

In [1]:
```
import json
import csv
import shutil
from datetime import datetime
from pathlib import Path

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import psycopg
from IPython.display import display, HTML

plt.rcParams['figure.figsize'] = (13, 6)
plt.rcParams['font.size'] = 11
plt.rcParams['axes.spines.top'] = False
plt.rcParams['axes.spines.right'] = False
sns.set_style('whitegrid')

WATERMARK = 'Vikrant Potnis | vikrant.potnis@gmail.com | Prepared for Aziro

# ── Paths ──────────────────────────────────────────────────────────────
_cwd = Path.cwd()
for _base in [_cwd, _cwd.parent]:
    if (_base / 'dataset').exists():
        DATASET    = _base / 'dataset'
        OUTPUTS    = _base / 'final_version' / 'outputs'
        OUTPUTS_NB = _base / 'final_version' / 'outputs_notebook'
        break
else:
    raise RuntimeError(f'Cannot find dataset/ from {_cwd}')

# Clear outputs_notebook/ so charts from previous runs don't accumulate
if OUTPUTS_NB.exists():
    shutil.rmtree(OUTPUTS_NB)
OUTPUTS_NB.mkdir()

# ── DB connection ──────────────────────────────────────────────────────
DSN = 'postgresql://rag_user:rag_pass@localhost:5434/rag_db'

def q(sql):
    """Run a SQL query and return a DataFrame."""
```

```
    with psycopg.connect(DSN) as conn:
        with conn.cursor() as cur:
            cur.execute(sql)
            cols = [d[0] for d in cur.description]
            return pd.DataFrame(cur.fetchall(), columns=cols)

def save(fig, name):
    """Save a figure to outputs_notebook/ with a watermark."""
    fig.text(0.5, 0.002, WATERMARK, ha='center', va='bottom',
             fontsize=6, color='#BBBBBB', alpha=0.8)
    fig.savefig(OUTPUTS_NB / name, dpi=150, bbox_inches='tight')
    print(f'Saved: {name}')

def label_bars(ax, fmt='{:.0f}', pad=0.3, horizontal=False):
    """Add value labels to every bar in a chart."""
    for bar in ax.patches:
        if horizontal:
            w = bar.get_width()
            if w > 0:
                ax.text(w + pad, bar.get_y() + bar.get_height() / 2,
                        fmt.format(w), va='center', fontsize=9)
        else:
            h = bar.get_height()
            if h > 0:
                ax.text(bar.get_x() + bar.get_width() / 2, h + pad,
                        fmt.format(h), ha='center', va='bottom', fontsize=9)

def stakeholder(ax, label):
    """Stamp a bold stakeholder label in the top-left corner of an axes."""
    ax.text(0.01, 1.02, f'For: {label}', transform=ax.transAxes,
            fontsize=9, fontweight='bold', color='white', va='bottom', ha='l
            bbox=dict(boxstyle='round,pad=0.3', facecolor='#444444', edgecol

def chart_header(audience, title):
    """Render a small bold header above a table output."""
    display(HTML(
        f'<p style="font-size:10px;margin:2px 0;"><b style="background:#444;
        f'color:#fff;padding:1px 5px;">For: {audience}</b></p>'
        f'<p style="font-size:12px;font-weight:bold;margin:2px 0 4px 0;">{ti
    ))

print('Ready.')
```

```
Ready.
```

---

2. Load Data into Postgres

Two steps:

- **Step 1** — read 100 meeting JSON folders → 6 base tables
- **Step 2** — read pre-computed clustering CSVs → 3 semantic tables + 1 view

Step 1 helpers — `create_base_tables` drops and recreates the 6 raw meeting tables. `load_base_data` reads every meeting folder in `dataset/` and inserts meetings, participants, summaries, key moments, action items, and transcript lines.
Step 2 helpers — `create_semantic_tables` creates the 3 clustering tables and the view. `load_semantic_data` reads the pre-computed CSVs and JSON from `final_version/outputs/`.

Database Schema

Data Model

In [2]:
```
# ── Constants ──────────────────────────────────────────────────────────
SCHEMA = 'meeting_analytics'
KNOWN_PRODUCTS = ['Detect', 'Protect', 'Comply', 'Identity']

# ── Small utilities ────────────────────────────────────────────────────
def parse_dt(s):
    # Convert ISO-8601 string (including Z suffix) to a Python datetime
    return datetime.fromisoformat(s.replace('Z', '+00:00')) if s else None

def parse_owner(raw):
    # Split "Owner Name: task description" into (owner, text)
    if ':' in raw:
        owner, _, text = raw.partition(':')
        return owner.strip(), text.strip()
    return '', raw.strip()

def extract_products(payload):
    # Scan summary text, action items, and key moments for known product nam
    text = ' '.join([
        payload.get('summary', ''),
        *payload.get('actionItems', []),
        *(km.get('text', '') for km in payload.get('keyMoments', []))
    ])
    return sorted(p for p in KNOWN_PRODUCTS if p in text)

# ── Step 1: base table DDL ─────────────────────────────────────────────
def create_base_tables(cur):
    cur.execute(f'DROP SCHEMA IF EXISTS {SCHEMA} CASCADE')   # wipe any prev
    cur.execute(f'CREATE SCHEMA {SCHEMA}')

    cur.execute(f'''
        CREATE TABLE {SCHEMA}.meetings (
            meeting_id      TEXT PRIMARY KEY,
            title           TEXT,
            organizer_email TEXT,
            duration_minutes NUMERIC,
            start_time      TIMESTAMPTZ
        )''')
```

```
    cur.execute(f'''
        CREATE TABLE {SCHEMA}.meeting_participants (
            meeting_id TEXT REFERENCES {SCHEMA}.meetings ON DELETE CASCADE,
            email      TEXT,
            PRIMARY KEY (meeting_id, email)
        )''')

    cur.execute(f'''
        CREATE TABLE {SCHEMA}.meeting_summaries (
            meeting_id        TEXT PRIMARY KEY REFERENCES {SCHEMA}.meetings
            summary_text      TEXT,
            overall_sentiment TEXT,
            sentiment_score   NUMERIC,
            topics            TEXT[],   -- pre-extracted topic phrases
            products          TEXT[]    -- detected product names
        )''')

    cur.execute(f'''
        CREATE TABLE {SCHEMA}.key_moments (
            meeting_id   TEXT REFERENCES {SCHEMA}.meetings ON DELETE CASCADE
            moment_index INTEGER,
            moment_type  TEXT,          -- churn_signal, praise, feature_gap
            text         TEXT,
            speaker      TEXT,
            time_seconds NUMERIC,
            PRIMARY KEY (meeting_id, moment_index)
        )''')

    cur.execute(f'''
        CREATE TABLE {SCHEMA}.action_items (
            meeting_id TEXT REFERENCES {SCHEMA}.meetings ON DELETE CASCADE,
            item_index INTEGER,
            owner      TEXT,
            text       TEXT,
            PRIMARY KEY (meeting_id, item_index)
        )''')

    cur.execute(f'''
        CREATE TABLE {SCHEMA}.transcript_lines (
            meeting_id     TEXT REFERENCES {SCHEMA}.meetings ON DELETE CASCA
            line_index     INTEGER,
            speaker        TEXT,
            sentence       TEXT,
            sentiment_type TEXT,
            time_seconds   NUMERIC,
            PRIMARY KEY (meeting_id, line_index)
        )''')

# ── Step 1: load raw JSONs ──────────────────────────────────────────
def load_base_data(cur, dataset_dir):
    meetings, participants, summaries, moments, actions, lines = [], [], [],

    for folder in sorted(dataset_dir.iterdir()):
        info_path = folder / 'meeting-info.json'
        if not info_path.exists():
            continue
```

```
        # Core meeting record
        info = json.loads(info_path.read_text())
        mid  = str(info.get('meetingId', folder.name))
        meetings.append((
            mid, info.get('title', ''),
            info.get('organizerEmail') or info.get('host', ''),
            info.get('duration'),
            parse_dt(info.get('startTime'))
        ))
        for email in info.get('allEmails', []):
            if email:
                participants.append((mid, email))

        # Summary, key moments, and action items (from summary.json)
        summary_path = folder / 'summary.json'
        if summary_path.exists():
            s = json.loads(summary_path.read_text())
            summaries.append((
                mid, s.get('summary', ''), s.get('overallSentiment'),
                s.get('sentimentScore'), s.get('topics', []), extract_produc
            ))
            for i, km in enumerate(s.get('keyMoments', [])):
                moments.append((mid, i, km.get('type'), km.get('text', ''),
                                km.get('speaker', ''), km.get('time')))
            for i, item in enumerate(s.get('actionItems', [])):
                owner, text = parse_owner(str(item))
                actions.append((mid, i, owner, text))

        # Sentence-level transcript (from transcript.json)
        transcript_path = folder / 'transcript.json'
        if transcript_path.exists():
            t = json.loads(transcript_path.read_text())
            for line in t.get('data', []):
                lines.append((
                    mid, int(line.get('index', 0)),
                    line.get('speaker_name', ''), line.get('sentence', ''),
                    line.get('sentimentType', ''), line.get('time')
                ))

    # Bulk insert all rows in one pass per table
    cur.executemany(f'INSERT INTO {SCHEMA}.meetings VALUES (%s,%s,%s,%s,%s)'
    cur.executemany(f'INSERT INTO {SCHEMA}.meeting_participants VALUES (%s,%
    cur.executemany(f'INSERT INTO {SCHEMA}.meeting_summaries VALUES (%s,%s,%
    cur.executemany(f'INSERT INTO {SCHEMA}.key_moments VALUES (%s,%s,%s,%s,%
    cur.executemany(f'INSERT INTO {SCHEMA}.action_items VALUES (%s,%s,%s,%s)
    cur.executemany(f'INSERT INTO {SCHEMA}.transcript_lines VALUES (%s,%s,%s

    return dict(meetings=len(meetings), participants=len(participants),
                summaries=len(summaries), key_moments=len(moments),
                action_items=len(actions), transcript_lines=len(lines))

# ── Step 2: semantic table DDL ──────────────────────────────────────
def create_semantic_tables(cur):
    cur.execute(f'''
        CREATE TABLE {SCHEMA}.semantic_clusters (
```

```
            cluster_id   INTEGER PRIMARY KEY,  -- assigned by HDBSCAN
            theme_title  TEXT,                 -- generated by LLM
            audience     TEXT,                 -- generated by LLM
            rationale    TEXT,                 -- generated by LLM
            phrase_count INTEGER
        )''')

    cur.execute(f'''
        CREATE TABLE {SCHEMA}.semantic_phrases (
            id         SERIAL PRIMARY KEY,
            canonical  TEXT UNIQUE,            -- deduplicated topic phrase
            cluster_id INTEGER REFERENCES {SCHEMA}.semantic_clusters
        )''')

    cur.execute(f'''
        CREATE TABLE {SCHEMA}.semantic_meeting_themes (
            meeting_id        TEXT,
            cluster_id        INTEGER REFERENCES {SCHEMA}.semantic_clusters,
            is_primary        BOOLEAN,          -- true for the dominant the
            call_type         TEXT,             -- support / external / inte
            sentiment_score   NUMERIC,
            overall_sentiment TEXT,
            products          TEXT[],           -- copied from meeting_summa
            PRIMARY KEY (meeting_id, cluster_id)
        )''')

    # View joining action items to their meeting's primary theme
    cur.execute(f'''
        CREATE VIEW {SCHEMA}.action_items_by_theme AS
        SELECT ai.meeting_id, ai.owner, ai.text AS action_item,
               smt.cluster_id, sc.theme_title, sc.audience
        FROM {SCHEMA}.action_items ai
        JOIN {SCHEMA}.semantic_meeting_themes smt
            ON ai.meeting_id = smt.meeting_id AND smt.is_primary = true
        JOIN {SCHEMA}.semantic_clusters sc ON smt.cluster_id = sc.cluster_ic
    ''')

# ── Step 2: load clustering outputs ──────────────────────────────────
def load_semantic_data(cur, outputs_dir):
    # Load cluster metadata from semantic_clusters.json
    clusters = json.loads((outputs_dir / 'semantic_clusters.json').read_text
    cur.executemany(
        f'INSERT INTO {SCHEMA}.semantic_clusters VALUES (%s,%s,%s,%s,%s)',
        [(c['cluster_id'], c['theme_title'], c['audience'],
          c.get('rationale', ''), c.get('phrase_count', 0)) for c in cluster
    )

    # Load phrase-to-cluster assignments from phrase_clusters.csv
    with open(outputs_dir / 'phrase_clusters.csv') as f:
        phrases = [(r['canonical'].strip(), int(r['cluster_id'])) for r in c
    cur.executemany(
        f'INSERT INTO {SCHEMA}.semantic_phrases (canonical, cluster_id) VALU
        phrases
    )

    # Load meeting-to-theme assignments from meeting_themes.csv
```

```
    # One row per (meeting, cluster) — a meeting can appear in multiple clus
    rows = []
    with open(outputs_dir / 'meeting_themes.csv') as f:
        for r in csv.DictReader(f):
            primary_id = int(r['primary_theme_id'])
            all_ids = [int(x) for x in r.get('all_theme_ids', '').split(';')
            score    = float(r['sentiment_score']) if r.get('sentiment_score
            for cid in all_ids:
                rows.append((r['meeting_id'], cid, cid == primary_id,
                             r.get('call_type', '') or 'unknown',
                             score, r.get('overall_sentiment') or None))
    cur.executemany(
        f'''INSERT INTO {SCHEMA}.semantic_meeting_themes
            (meeting_id, cluster_id, is_primary, call_type, sentiment_score,
            VALUES (%s,%s,%s,%s,%s,%s)''',
        rows
    )

    # Populate products column by copying from meeting_summaries
    cur.execute(f'''
        UPDATE {SCHEMA}.semantic_meeting_themes smt
        SET products = ms.products
        FROM {SCHEMA}.meeting_summaries ms
        WHERE smt.meeting_id = ms.meeting_id
    ''')

    return dict(clusters=len(clusters), phrases=len(phrases), meeting_themes

print('Helpers defined.')
```

```
Helpers defined.
```

Run the load. Drops and recreates the schema, inserts all raw meeting data, then loads the clustering outputs. All 9 tables are populated in a single transaction.

In [3]:
```
with psycopg.connect(DSN) as conn:
    with conn.cursor() as cur:
        print('Step 1 — creating base tables...')
        create_base_tables(cur)

        print('Step 1 — loading raw meeting JSONs...')
        base_counts = load_base_data(cur, DATASET)
        for table, n in base_counts.items():
            print(f'  {table:<20} {n}')

        print('Step 2 — creating semantic tables...')
        create_semantic_tables(cur)

        print('Step 2 — loading clustering outputs...')
        sem_counts = load_semantic_data(cur, OUTPUTS)
        for table, n in sem_counts.items():
            print(f'  {table:<20} {n}')

print('Done.')
```

```
Step 1 — creating base tables...
Step 1 — loading raw meeting JSONs...
  meetings             100
  participants         311
  summaries            100
  key_moments          402
  action_items         397
  transcript_lines     4313
Step 2 — creating semantic tables...
Step 2 — loading clustering outputs...
  clusters             26
  phrases              343
  meeting_themes       516
Done.
```

Verify. Checks exact row counts for all 9 tables against known expected values. All should show PASS before proceeding.

In [4]:
```
# Quick sanity check — expected row counts
expected = {
    'meetings': 100, 'meeting_participants': 311, 'meeting_summaries': 100,
    'key_moments': 402, 'action_items': 397, 'transcript_lines': 4313,
    'semantic_clusters': 26, 'semantic_phrases': 343, 'semantic_meeting_them
}
for table, exp in expected.items():
    actual = q(f'SELECT count(*) FROM meeting_analytics.{table}').iloc[0, 0]
    status = 'PASS' if actual == exp else 'FAIL'
    print(f'{status}  {table:<30} {actual} (expected {exp})')
```

```
PASS  meetings                      100 (expected 100)
PASS  meeting_participants          311 (expected 311)
PASS  meeting_summaries             100 (expected 100)
PASS  key_moments                   402 (expected 402)
PASS  action_items                  397 (expected 397)
PASS  transcript_lines              4313 (expected 4313)
PASS  semantic_clusters             26 (expected 26)
PASS  semantic_phrases              343 (expected 343)
PASS  semantic_meeting_themes       516 (expected 516)
```

Chart Summary

#	Chart	Description	For
3.1	Meeting Breakdown by Call Type and Product	Call type split and product mentions across 100 meetings	All
4.1	UMAP Scatter	343 topic phrases mapped to 2D — 26 clusters visible as distinct islands	All
4.2	26 Themes Table	All discovered themes ranked by number of meetings	All
5.1	Theme Sentiment Heatmap	Themes sorted most negative (top) to most positive — Detect is red,	All Leadership

#	Chart	Description	For
		Comply is green	
6.1	Churn Signals by Product	Total explicit churn signals per product	**Sales, CS**
6.2	Accounts Needing Immediate Follow-Up	38 meetings with negative sentiment and churn signals, ranked by urgency	**Sales, CS**
6.3	Which Products Are Generating the Most Risk?	Total meetings, technical issues, and churn signals per product	**Engineering, Product**
6.4	Where Customers Are Happy	Praise density by product + Comply renewal sentiment	**Product, Marketing, Sales, CS**
7.1	How the Detect Outage Affected Renewal Calls	Sentiment split for Detect vs non-Detect renewal meetings	**Engineering, Sales**
7.2	What Customers Are Asking For and How Urgently	Blocked (red) = at risk, act now. Growing (green) = healthy account, roadmap ask	**Product (CPO)**
7.3	Who Owns the Follow-Up Work?	Follow-up actions by theme and by department across products	**Operations, Engineering, CS, Sales**

---

3. Dataset Overview

3.1 Meeting Breakdown by Call Type and Product

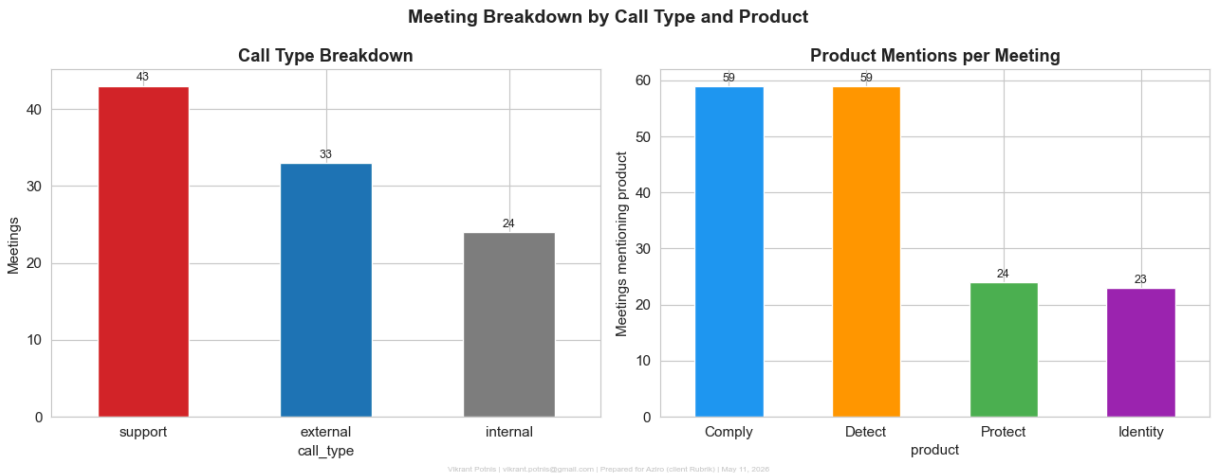
For: All

Call type split and product mentions across the 100 meetings — sets the scale before the analysis.

In [5]:
```
# Query call type split and product mentions
call_types = q('''
    SELECT call_type, count(DISTINCT meeting_id) AS meetings
    FROM meeting_analytics.semantic_meeting_themes
    WHERE is_primary = true
    GROUP BY call_type ORDER BY meetings DESC
''')
products = q('''
    SELECT unnest(products) AS product, count(*) AS meetings
    FROM meeting_analytics.meeting_summaries
    GROUP BY product ORDER BY meetings DESC
''')

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(13, 5))

# Left: call type breakdown
```

```
call_types.set_index('call_type')['meetings'].plot.bar(
    ax=ax1, color=['#d62728', '#1f77b4', '#7f7f7f'], width=0.5, rot=0)
label_bars(ax1)
ax1.set_title('Call Type Breakdown', fontweight='bold')
ax1.set_ylabel('Meetings')

# Right: product mentions
products.set_index('product')['meetings'].plot.bar(
    ax=ax2, color=['#2196F3', '#FF9800', '#4CAF50', '#9C27B0'], width=0.5, r
label_bars(ax2)
ax2.set_title('Product Mentions per Meeting', fontweight='bold')
ax2.set_ylabel('Meetings mentioning product')

fig.suptitle('Meeting Breakdown by Call Type and Product', fontsize=14, font
plt.tight_layout()
save(fig, '00_dataset_overview.png')
plt.show()
```

```
Saved: 00_dataset_overview.png
```



---

4. Theme Classification

Approach: embed topic phrases → HDBSCAN clustering → LLM labeling. 26 themes found naturally — no fixed K required.

4.1 UMAP Scatter

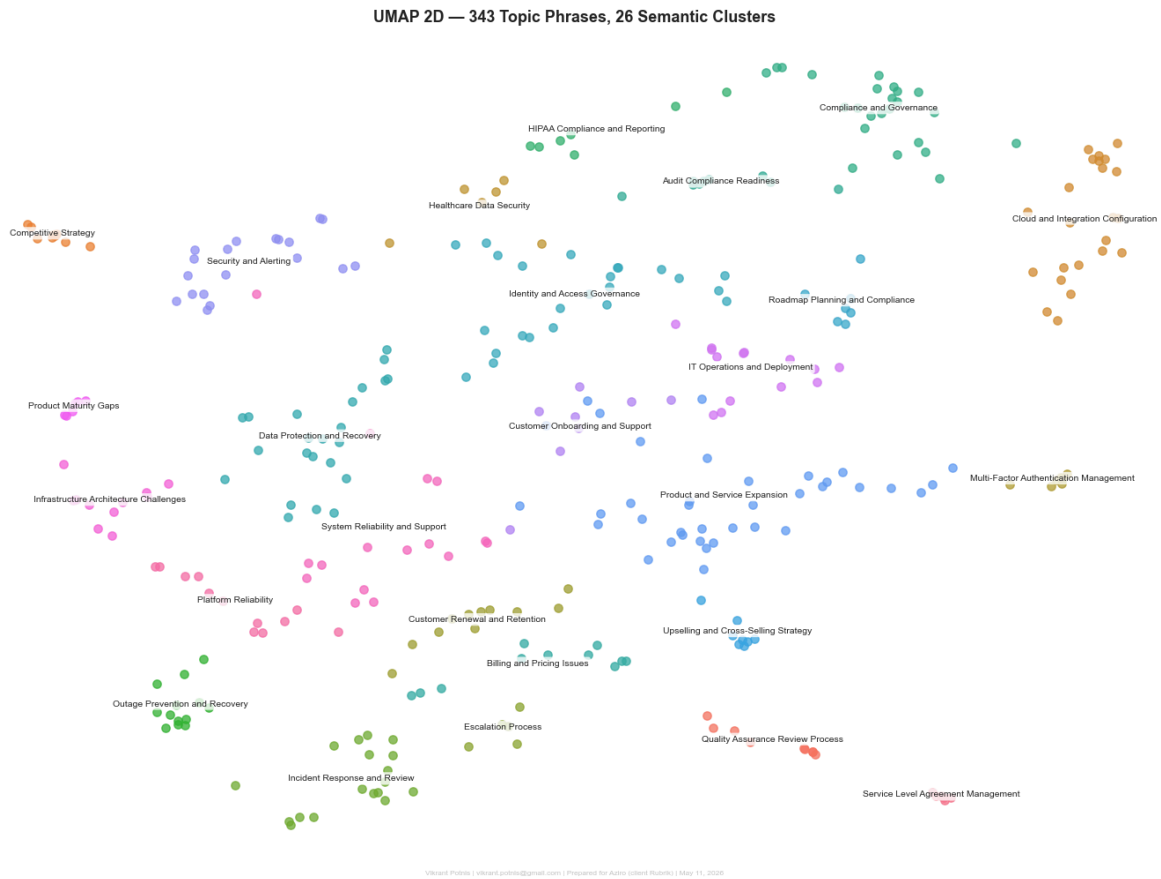
For: All

343 topic phrases reduced to 2D. Each point is a phrase, colored by cluster. Labels show the theme name at each cluster's centroid. Clustering ran on 10-dim UMAP — this 2D view is for visualisation only.

In [6]:
```
# Load pre-computed 2D UMAP coordinates (separate from the 10-dim clustering
coords = pd.read_csv(OUTPUTS / 'viz_coords.csv')

# Assign a distinct colour to each cluster
```

```
palette = sns.color_palette('husl', coords['cluster_id'].nunique())
color_map = {cid: palette[i] for i, cid in enumerate(sorted(coords['cluster_

fig, ax = plt.subplots(figsize=(14, 10))

# Plot each cluster's phrases as a scatter group
for cid, grp in coords.groupby('cluster_id'):
    ax.scatter(grp['x'], grp['y'], c=[color_map[cid]], s=45, alpha=0.75)

# Label each cluster at its centroid
for _, row in coords.groupby(['cluster_id', 'theme_title'])[['x', 'y']].mear
    ax.annotate(row['theme_title'], (row['x'], row['y']), fontsize=7.5, ha='
                bbox=dict(boxstyle='round,pad=0.25', facecolor='white', alph

ax.set_title('UMAP 2D — 343 Topic Phrases, 26 Semantic Clusters', fontsize=1
ax.axis('off')
plt.tight_layout()
save(fig, '01_umap_scatter.png')
plt.show()
```

```
Saved: 01_umap_scatter.png
```



4.2 26 Themes Table

For: All

All discovered themes ranked by primary meetings dominated, with target audience and phrase count.

In [7]:
```
# Query all 26 themes with topic count and number of meetings each one domir
clusters = q('''
    SELECT sc.theme_title, sc.audience, sc.phrase_count,
           count(DISTINCT smt.meeting_id) AS primary_meetings
    FROM meeting_analytics.semantic_clusters sc
    JOIN meeting_analytics.semantic_meeting_themes smt
        ON sc.cluster_id = smt.cluster_id AND smt.is_primary = true
    GROUP BY sc.theme_title, sc.audience, sc.phrase_count
    ORDER BY primary_meetings DESC
''')
clusters.columns = ['Theme', 'Audience', 'Topics', 'Meetings']

# Display as a styled table with a bar indicator on the meeting count colum
display(
    clusters.style
    .bar(subset=['Meetings'], color='#1f77b4')
    .set_caption('26 themes ranked by number of meetings')
    .hide(axis='index')
)
```

26 themes ranked by number of meetings

Theme	Audience	Topics	Meetings
Product and Service Expansion	All	37	14
Outage Prevention and Recovery	Engineering	13	11
Customer Renewal and Retention	Sales | Customer Success	11	11
Compliance and Governance	All	24	9
IT Operations and Deployment	Engineering	14	8
Data Protection and Recovery	All	23	6
Incident Response and Review	All	18	6
System Reliability and Support	All	16	5
Billing and Pricing Issues	Sales | Customer Support	11	5
Identity and Access Governance	All	24	4
Competitive Strategy	Sales	9	3
Security and Alerting	All	19	3
Cloud and Integration Configuration	Engineering	23	3
HIPAA Compliance and Reporting	All	7	2
Roadmap Planning and Compliance	All	7	2
Audit Compliance Readiness	All	9	2
Customer Onboarding and Support	All	8	1
Product Maturity Gaps	All	6	1
Quality Assurance Review Process	All	9	1
Healthcare Data Security	All	6	1
Multi-Factor Authentication Management	Engineering	7	1
Platform Reliability	All	12	1

---

5. Sentiment Analysis

5.1 Theme Sentiment Heatmap

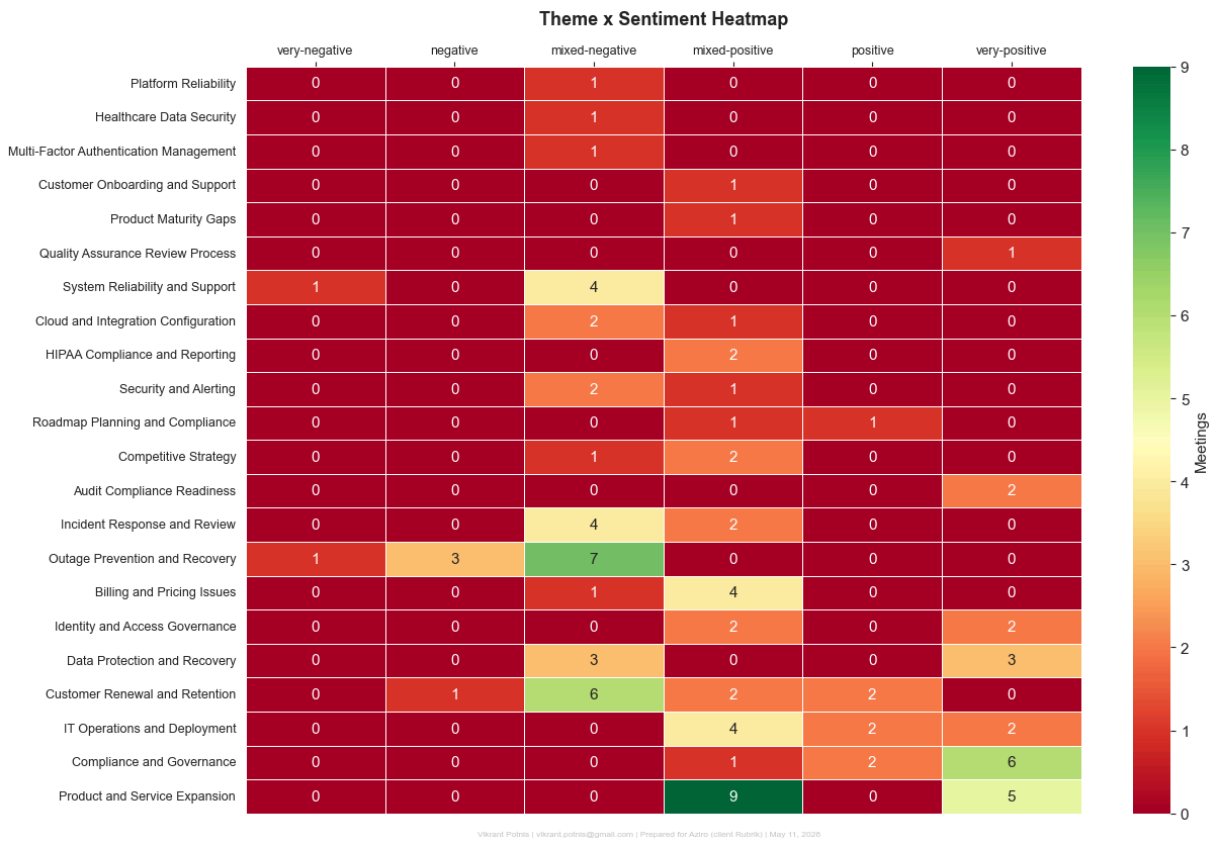
For: All Leadership

Every row is a theme, every column a sentiment bucket. Themes are sorted most

negative (top) to most positive (bottom). Each cell shows the number of meetings at that combination.

In [8]:
```
# Query meeting count for every (theme, sentiment) combination
hm = q('''
    SELECT sc.theme_title, smt.overall_sentiment, count(*) AS meetings
    FROM meeting_analytics.semantic_meeting_themes smt
    JOIN meeting_analytics.semantic_clusters sc ON smt.cluster_id = sc.clust
    WHERE smt.is_primary = true AND smt.overall_sentiment IS NOT NULL
    GROUP BY sc.theme_title, smt.overall_sentiment
''')

# Build pivot table: themes as rows, sentiment buckets as columns (in severi
sent_order = ['very-negative', 'negative', 'mixed-negative', 'neutral',
              'mixed-positive', 'positive', 'very-positive']
pivot = hm.pivot_table(index='theme_title', columns='overall_sentiment',
                       values='meetings', fill_value=0)
pivot = pivot.reindex(columns=[c for c in sent_order if c in pivot.columns])

# Sort rows from most negative (top) to most positive (bottom)
weights = {s: i for i, s in enumerate(sent_order)}
pivot['_score'] = sum(pivot.get(c, 0) * weights.get(c, 3) for c in sent_orde
pivot = pivot.sort_values('_score').drop(columns=['_score']).astype(int)

# Draw annotated heatmap
fig, ax = plt.subplots(figsize=(13, max(8, len(pivot) * 0.38)))
sns.heatmap(pivot, annot=True, fmt='d', cmap='RdYlGn',
            linewidths=0.4, linecolor='white', ax=ax,
            cbar_kws={'label': 'Meetings'})

# Move column labels to the top
ax.xaxis.tick_top()
ax.xaxis.set_label_position('top')

ax.set_title('Theme x Sentiment Heatmap', fontsize=13, fontweight='bold', pa
ax.set_xlabel('')
ax.set_ylabel('')
ax.tick_params(axis='y', labelsize=9)
ax.tick_params(axis='x', labelsize=9)
plt.tight_layout()
save(fig, '03_theme_sentiment_heatmap.png')
plt.show()
```

```
Saved: 03_theme_sentiment_heatmap.png
```



---

6. Additional Insights

#	Insight	Who acts on it
6.1	Churn signals by product	Sales · CS
6.2	High-risk account watchlist	CSMs · AEs
6.3	Technical issues & churn signals by product	Engineering · CTO
6.4	Positive signals — where we're winning	Product · Marketing

6.1 Churn Signals by Product

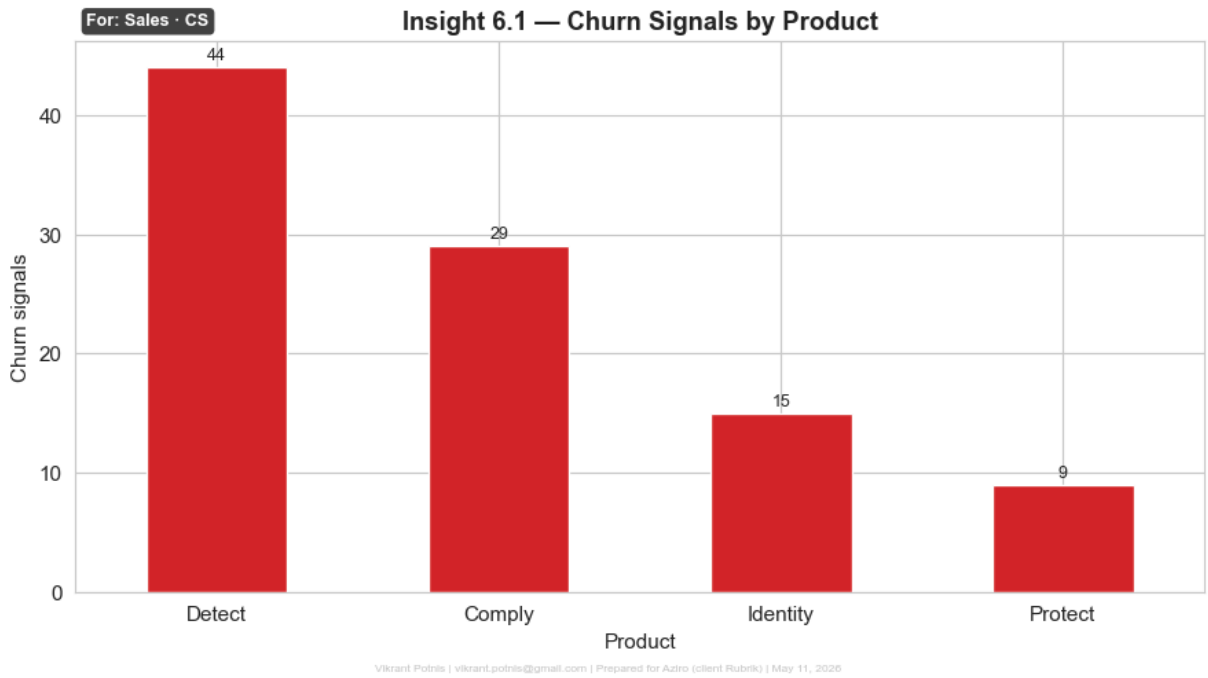
For: Sales · CS

Total number of explicit churn signals per product across all meetings. Shows which product is most associated with at-risk conversations.

In [9]:
```
# Query total churn signals per product across all meetings
churn_by_product = q('''
    SELECT p.product, count(km.moment_index) AS churn_signals
    FROM (SELECT DISTINCT unnest(products) AS product, meeting_id
          FROM meeting_analytics.meeting_summaries WHERE products <> \'{}\')
    JOIN meeting_analytics.key_moments km
        ON p.meeting_id = km.meeting_id AND km.moment_type = \'churn_signal\
```

```
    GROUP BY p.product ORDER BY churn_signals DESC
''')

fig, ax = plt.subplots(figsize=(9, 5))

# One bar per product, sorted by churn signal count
churn_by_product.set_index('product')['churn_signals'].plot.bar(
    ax=ax, color='#d62728', width=0.5, rot=0)
label_bars(ax)
stakeholder(ax, 'Sales · CS')
ax.set_title('Insight 6.1 — Churn Signals by Product', fontweight='bold')
ax.set_ylabel('Churn signals')
ax.set_xlabel('Product')
plt.tight_layout()
save(fig, '04_churn_by_product.png')
plt.show()
```

```
Saved: 04_churn_by_product.png
```



6.2 Accounts Needing Immediate Follow-Up

For: Sales, CS

Meetings with negative sentiment and at least one explicit churn signal, ranked by signal count. These accounts need a follow-up call this week.

In [10]:
```
# Query meetings that have both negative sentiment AND at least one churn si
watchlist = q('''
    SELECT m.title, sc.theme_title AS primary_theme,
           smt.call_type, smt.overall_sentiment,
           round(smt.sentiment_score::numeric, 1) AS score,
           count(km.moment_index) AS churn_signals
    FROM meeting_analytics.meetings m
```

```
    JOIN meeting_analytics.semantic_meeting_themes smt
        ON m.meeting_id = smt.meeting_id AND smt.is_primary = true
    JOIN meeting_analytics.semantic_clusters sc ON smt.cluster_id = sc.clust
    JOIN meeting_analytics.key_moments km
        ON m.meeting_id = km.meeting_id AND km.moment_type = \'churn_signal\
    WHERE smt.overall_sentiment IN (\'negative\',\'very-negative\',\'mixed-r
    GROUP BY m.title, sc.theme_title,
             smt.call_type, smt.overall_sentiment, smt.sentiment_score
    ORDER BY churn_signals DESC, smt.sentiment_score ASC
''')

# Render a styled header before the table
chart_header('Sales, CS',
             f'Accounts Needing Immediate Follow-Up — {len(watchlist)} meeti

watchlist.columns = ['Title', 'Primary Theme', 'Call Type', 'Sentiment', 'Sc

# Display as styled table — red gradient on churn signals, green/red on scor
display(
    watchlist.style
    .background_gradient(subset=['Churn Signals'], cmap='Reds')
    .background_gradient(subset=['Score'], cmap='RdYlGn')
    .hide(axis='index')
)

# Save top 15 rows as a PNG for the slide deck
top = watchlist.head(15).reset_index(drop=True)
col_widths = [0.32, 0.22, 0.1, 0.14, 0.08, 0.14]
fig, ax = plt.subplots(figsize=(13, max(4, len(top) * 0.38 + 0.8)))
ax.axis('off')
tbl = ax.table(
    cellText=top.values,
    colLabels=top.columns,
    cellLoc='left',
    loc='center',
    colWidths=col_widths
)
tbl.auto_set_font_size(False)
tbl.set_fontsize(8)
# Style header row
for j in range(len(top.columns)):
    tbl[0, j].set_facecolor('#444444')
    tbl[0, j].set_text_props(color='white', fontweight='bold')
# Colour churn signals column by value
max_churn = top['Churn Signals'].max()
for i in range(len(top)):
    v = top.iloc[i]['Churn Signals']
    intensity = int(255 - (v / max_churn) * 160)
    tbl[i + 1, 5].set_facecolor((1, intensity/255, intensity/255))
    tbl[i + 1, 4].set_facecolor(
        (intensity/255, 1, intensity/255) if float(top.iloc[i]['Score']) >=
        else (1, intensity/255, intensity/255)
    )
plt.tight_layout()
save(fig, '04b_watchlist.png')
plt.close()
```

For: Sales, CS

Accounts Needing Immediate Follow-Up — 38 meetings identified

Title	Primary Theme	Call Type	Sentiment	Score	Churn Signals
Support Case #3266 - Trailhead Marketplace Detect Alerts Not Firing	Outage Prevention and Recovery	support	very-negative	1.4	2
Detect Outage - Customer Impact Assessment	Outage Prevention and Recovery	internal	negative	1.8	2
Aegis / Northstar Pharma - Urgent: Detect Outage Impact	Customer Renewal and Retention	support	mixed-negative	2.1	2
Support Case #3677 - Coastal Living Co MFA Token Failures	Multi-Factor Authentication Management	support	mixed-negative	2.2	2
Aegis / Summit Trust - Platform Concerns Discussion	Customer Renewal and Retention	support	mixed-negative	2.4	2
Support Case #7615 - Crestline Wealth Group Policy Sync Delay	System Reliability and Support	support	mixed-negative	2.4	2
Aegis / Nova Retail Group - Renewal Discussion	Customer Renewal and Retention	external	mixed-negative	2.6	2
Aegis / Ironclad Financial - Account Recovery Check-in	Incident Response and Review	support	mixed-negative	2.8	2
URGENT: Blackridge Investments - Complete Loss of Threat Visibility	System Reliability and Support	support	very-negative	1.6	1
INCIDENT: Detect Pipeline Failure - War Room	Outage Prevention and Recovery	support	negative	1.8	1
Detect Outage - Escalation Bridge	Customer Renewal and Retention	support	negative	1.8	1
URGENT: Cobalt Software - Aegis Detect Dashboard Down	Outage Prevention and Recovery	support	negative	1.8	1
Support Case #6573 - Ridgeline Logistics Detect Data Gaps	Outage Prevention and Recovery	support	mixed-negative	2.1	1
ESCALATION: Northstar Pharma - Detect Outage Impact on Compliance	Outage Prevention and Recovery	support	mixed-negative	2.1	1
Protect Performance - Scalability Concerns	Data Protection and Recovery	support	mixed-negative	2.2	1
Support Case #8749 - Coastal Living Co SAML Certificate Rotation Bug	Cloud and Integration Configuration	support	mixed-negative	2.2	1

Title	Primary Theme	Call Type	Sentiment	Score	Churn Signals
Support Case #6635 - Nova Retail Group SSO Login Failures	Cloud and Integration Configuration	support	mixed-negative	2.3	1
Competitive Threat Assessment - Post Outage	Outage Prevention and Recovery	support	mixed-negative	2.3	1
Aegis / Helix Data - Post-Incident Review	Incident Response and Review	support	mixed-negative	2.3	1
Win/Loss Analysis - Q1	Competitive Strategy	external	mixed-negative	2.4	1
Aegis / Meridian Capital - Service Reliability Discussion	Outage Prevention and Recovery	support	mixed-negative	2.4	1
Aegis / Quantum Edge - Renewal Concerns	Outage Prevention and Recovery	support	mixed-negative	2.4	1
Detect Outage - Remediation Plan Review	Incident Response and Review	support	mixed-negative	2.4	1
Support Case #1750 - Vanta Health Systems SIEM Connector Issue	System Reliability and Support	support	mixed-negative	2.4	1
Support Case #3103 - Silverline Brands Detect False Positives After Patch	Healthcare Data Security	support	mixed-negative	2.4	1
Support Case #3286 - Vanta Health Systems API Integration Error	Outage Prevention and Recovery	support	mixed-negative	2.4	1
Support Case #3536 - Stratos Cloud LogVault Integration Timeout	System Reliability and Support	support	mixed-negative	2.4	1
Support Case #5889 - Ridgeline Logistics Detect Latency Issues	Security and Alerting	support	mixed-negative	2.4	1
Support Case #6147 - Summit Trust Detect Alert Delays	Security and Alerting	support	mixed-negative	2.4	1
Support Case #6539 - Steelpoint Manufacturing LDAP Sync Failures	System Reliability and Support	support	mixed-negative	2.4	1
Support Case #8179 - Clearwater Medical Overage Charges Dispute	Billing and Pricing Issues	support	mixed-negative	2.4	1
Support Case #9856 - Silverline Brands Backup Window Exceeded	Data Protection and Recovery	support	mixed-negative	2.4	1

	Title	Primary Theme	Call Type	Sentiment	Score	Churn Signals
	Aegis / Ironworks Corp - Vendor Comparison Discussion	Outage Prevention and Recovery	support	mixed-negative	2.6	1
	Aegis / Brightpath Commerce - Competitive Evaluation	Customer Renewal and Retention	support	mixed-negative	2.6	1
	Detect Outage - Post-Incident Review	Customer Renewal and Retention	internal	mixed-negative	2.8	1
	Aegis / Steelpoint Manufacturing - Contract Review	Customer Renewal and Retention	external	mixed-negative	2.8	1
	Support Case #6977 - Brightpath Commerce Slow Backup Performance	Data Protection and Recovery	support	mixed-negative	2.8	1
	Detect Reliability - 30-Day Review	Incident Response and Review	internal	mixed-negative	2.8	1

```
Saved: 04b_watchlist.png
```

6.3 Which Products Are Generating the Most Risk

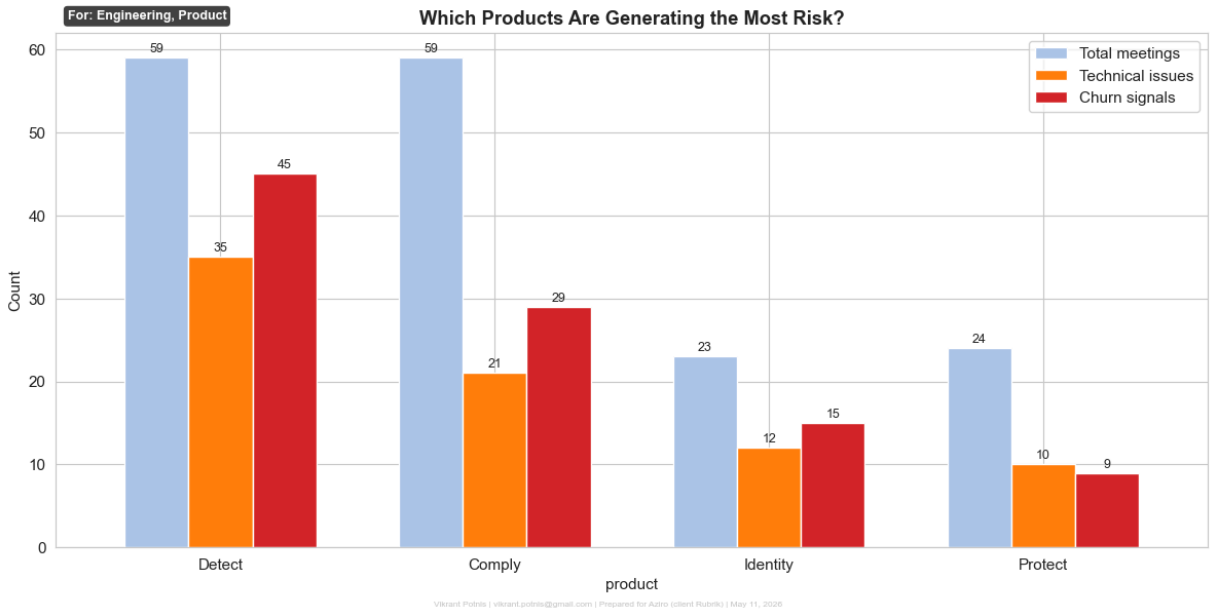
For: Engineering, Product

Three bars per product: total meetings, technical issues raised, and churn signals. Shows which product is driving the most support load and commercial risk.

In [11]:
```
# Query total meetings, technical issues, and churn signals per product
prod_sig = q('''
    SELECT p.product,
           count(DISTINCT p.meeting_id) AS total_meetings,
           count(km_t.moment_index) AS tech_issues,
           count(km_c.moment_index) AS churn_signals
    FROM (SELECT DISTINCT unnest(products) AS product, meeting_id
          FROM meeting_analytics.meeting_summaries WHERE products <> \'{}\')
    LEFT JOIN meeting_analytics.key_moments km_t
        ON p.meeting_id = km_t.meeting_id AND km_t.moment_type = \'technical
    LEFT JOIN meeting_analytics.key_moments km_c
        ON p.meeting_id = km_c.meeting_id AND km_c.moment_type = \'churn_sig
    GROUP BY p.product ORDER BY tech_issues DESC
''')

fig, ax = plt.subplots(figsize=(12, 6))

# Grouped bar: 3 bars per product (total meetings, tech issues, churn signal
prod_sig.set_index('product')[['total_meetings', 'tech_issues', 'churn_signa
    ax=ax, color=['#aec7e8', '#ff7f0e', '#d62728'], width=0.7, rot=0)
label_bars(ax)
```

```
stakeholder(ax, 'Engineering, Product')
ax.set_title('Which Products Are Generating the Most Risk?', fontweight='bol
ax.set_ylabel('Count')
ax.legend(['Total meetings', 'Technical issues', 'Churn signals'])
plt.tight_layout()
save(fig, '05_product_signals.png')
plt.show()
```

```
Saved: 05_product_signals.png
```



6.4 Where Customers Are Happy

For: Product, Marketing, Sales, CS

Left: praise moments by product — where customers are happy. Right: Comply external meeting sentiment — the good news story to pair with the Detect recovery message.

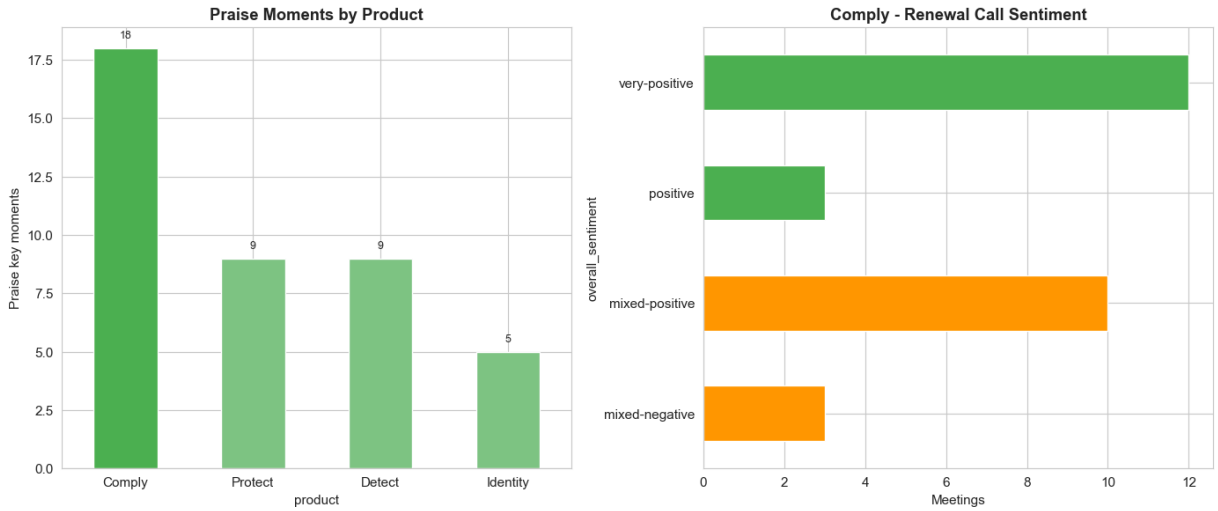
In [12]:
```
# Query praise moments per product and Comply external meeting sentiment
praise = q('''
    SELECT p.product, count(km.moment_index) AS praise_moments
    FROM (SELECT DISTINCT unnest(products) AS product, meeting_id
          FROM meeting_analytics.meeting_summaries WHERE products <> \'{}\')
    JOIN meeting_analytics.key_moments km
        ON p.meeting_id = km.meeting_id AND km.moment_type = \'praise\'
    GROUP BY p.product ORDER BY praise_moments DESC
''')
comply_ext = q('''
    SELECT overall_sentiment, count(*) AS meetings
    FROM meeting_analytics.semantic_meeting_themes
    WHERE \'Comply\' = ANY(products) AND call_type = \'external\' AND is_pri
    GROUP BY overall_sentiment ORDER BY meetings DESC
''')

# Sort comply sentiment rows in severity order (most negative first)
sent_order = ['very-negative','negative','mixed-negative','neutral','mixed-p
```

```
comply_ext['_r'] = comply_ext['overall_sentiment'].map({s: i for i, s in enu
comply_ext = comply_ext.sort_values('_r').drop(columns=['_r'])

# Colour each bar by sentiment (red = negative, orange = mixed, green = posi
sent_colors = ['#d62728' if 'negative' in s and 'mixed' not in s
               else '#ff9800' if 'mixed' in s else '#4CAF50' if 'positive' i
               else '#9E9E9E' for s in comply_ext['overall_sentiment']]

chart_header('Product, Marketing, Sales, CS', 'Where Customers Are Happy')

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 6))

# Left: praise moments bar chart
colors_pr = ['#4CAF50' if p == 'Comply' else '#81C784' for p in praise['proc
praise.set_index('product')['praise_moments'].plot.bar(ax=ax1, color=colors_
label_bars(ax1)
ax1.set_title('Praise Moments by Product', fontweight='bold')
ax1.set_ylabel('Praise key moments')

# Right: Comply external meeting sentiment (horizontal bar)
comply_ext.set_index('overall_sentiment')['meetings'].plot.barh(ax=ax2, colc
ax2.set_title('Comply - Renewal Call Sentiment', fontweight='bold')
ax2.set_xlabel('Meetings')

plt.tight_layout()
save(fig, '06_positive_signals.png')
plt.show()
```

For: Product, Marketing, Sales, CS

Where Customers Are Happy

```
Saved: 06_positive_signals.png
```



---

7. Leadership Questions

Chart	Question	Stakeholder
E3/R4	How many external meetings were contaminated by the Detect outage?	CTO · Sales
P1	Feature gaps by product × sentiment	CPO
S3	Action item owners and workload	Operations

7.1 How the Detect Outage Affected Renewal Calls

For: Engineering, Sales

Left: sentiment split for renewal meetings that mention Detect vs those that don't. Right: what fraction of Detect renewal meetings carry an explicit churn signal.

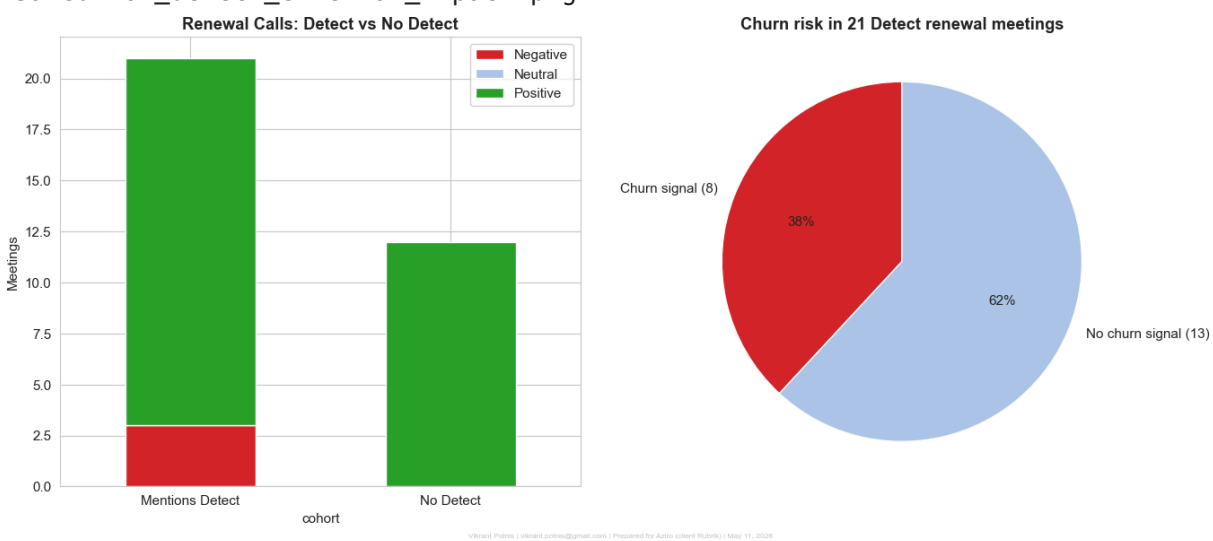
In [13]:
```
# Query external meeting sentiment split: Detect-tagged vs Detect-free
ext_detect = q('''
    SELECT
        CASE WHEN \'Detect\' = ANY(smt.products) THEN \'Mentions Detect\' EL
        CASE
            WHEN smt.overall_sentiment IN (\'negative\',\'very-negative\',\'
            WHEN smt.overall_sentiment = \'neutral\' THEN \'Neutral\'
            ELSE \'Positive\'
        END AS bucket,
        count(*) AS meetings
    FROM meeting_analytics.semantic_meeting_themes smt
    WHERE smt.call_type = \'external\' AND smt.is_primary = true
    GROUP BY cohort, bucket
''')

# Query how many Detect-tagged external meetings have a churn signal
churn_detect = q('''
    SELECT count(DISTINCT km.meeting_id) AS n
    FROM meeting_analytics.key_moments km
    JOIN meeting_analytics.semantic_meeting_themes smt
        ON km.meeting_id = smt.meeting_id AND smt.is_primary = true
    WHERE smt.call_type = \'external\' AND \'Detect\' = ANY(smt.products)
      AND km.moment_type = \'churn_signal\'
''')

# Build pivot for stacked bar (cohort x sentiment bucket)
pv = ext_detect.pivot_table(index='cohort', columns='bucket', values='meetir
for col in ['Negative', 'Neutral', 'Positive']:
    if col not in pv.columns: pv[col] = 0
pv = pv[['Negative', 'Neutral', 'Positive']]

churn_n    = int(churn_detect.iloc[0]['n'])
detect_tot = int(ext_detect[ext_detect['cohort'] == 'Mentions Detect']['meet

chart_header('Engineering, Sales', 'How the Detect Outage Affected Renewal C

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 6))
```

```
# Left: stacked bar — sentiment split by cohort
pv.plot.bar(stacked=True, ax=ax1,
            color=['#d62728', '#aec7e8', '#2ca02c'], width=0.5, rot=0)
ax1.set_title('Renewal Calls: Detect vs No Detect', fontweight='bold')
ax1.set_ylabel('Meetings')
ax1.legend()

# Right: pie chart — churn signal rate in Detect renewal meetings
ax2.pie(
    [churn_n, detect_tot - churn_n],
    labels=[f'Churn signal ({churn_n})', f'No churn signal ({detect_tot - ch
    colors=['#d62728', '#aec7e8'], startangle=90, autopct='%1.0f%%'
)
ax2.set_title(f'Churn risk in {detect_tot} Detect renewal meetings', fontwei

plt.tight_layout()
save(fig, '07_detect_external_impact.png')
plt.show()
print(f'{churn_n} of {detect_tot} Detect renewal meetings have explicit chur
```

For: Engineering, Sales

How the Detect Outage Affected Renewal Calls

```
Saved: 07_detect_external_impact.png
```




```
8 of 21 Detect renewal meetings have explicit churn signals.
```

7.2 What Customers Are Asking For and How Urgently

For: Product (CPO)

Same feature request carries different urgency. Blocked (red) = customer is frustrated and at risk — treat as P0. Growing (green) = expanding account asking for more — roadmap investment.

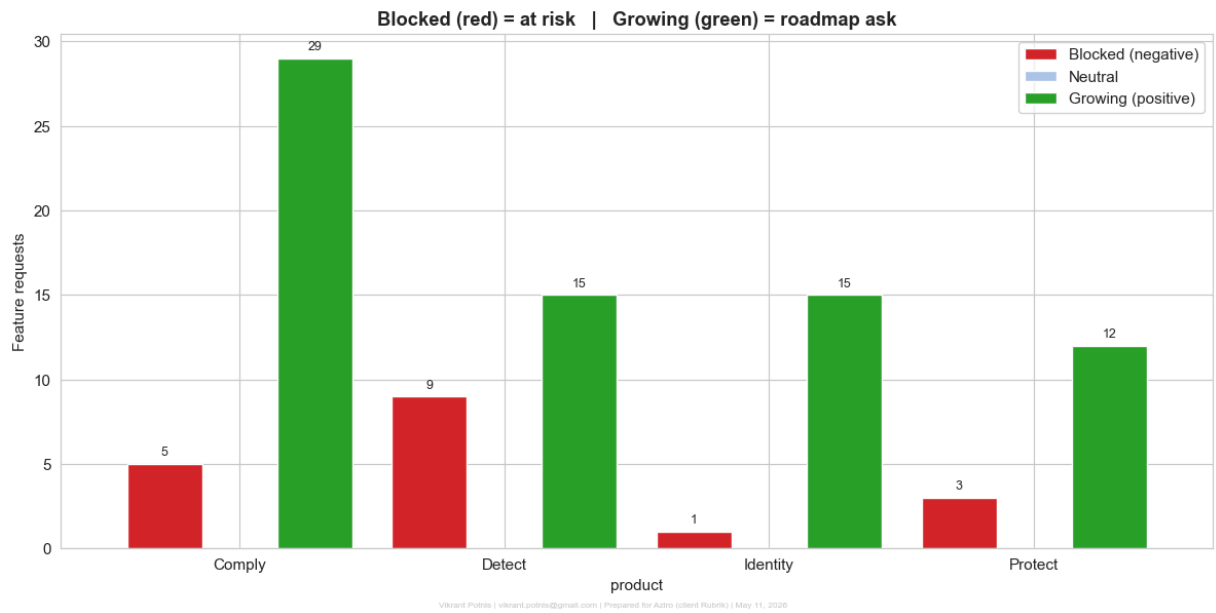
In [14]:
```
# Query feature gap count per product, split by customer sentiment at time c
feat = q('''
    SELECT p.product,
```

```
        CASE
            WHEN ms.overall_sentiment IN (\'negative\',\'very-negative\',\'m
            WHEN ms.overall_sentiment = \'neutral\' THEN \'Neutral\'
            ELSE \'Growing (positive)\'
        END AS sentiment_bucket,
        count(km.moment_index) AS feature_gaps
    FROM (SELECT DISTINCT unnest(products) AS product, meeting_id
          FROM meeting_analytics.meeting_summaries WHERE products <> \'{}\')
    JOIN meeting_analytics.key_moments km
        ON p.meeting_id = km.meeting_id AND km.moment_type = \'feature_gap\'
    JOIN meeting_analytics.meeting_summaries ms ON p.meeting_id = ms.meeting
    GROUP BY p.product, sentiment_bucket
''')

# Build pivot: products as rows, sentiment buckets as columns
pv_feat = feat.pivot_table(index='product', columns='sentiment_bucket',
                           values='feature_gaps', fill_value=0)
for col in ['Blocked (negative)', 'Neutral', 'Growing (positive)']:
    if col not in pv_feat.columns: pv_feat[col] = 0
pv_feat = pv_feat[['Blocked (negative)', 'Neutral', 'Growing (positive)']]

chart_header('Product (CPO)', 'What Customers Are Asking For and How Urgentl

fig, ax = plt.subplots(figsize=(12, 6))

# Grouped bar: 3 bars per product (blocked / neutral / growing)
pv_feat.plot.bar(ax=ax, color=['#d62728', '#aec7e8', '#2ca02c'], width=0.85,
label_bars(ax)
ax.set_title('Blocked (red) = at risk   |   Growing (green) = roadmap ask',
ax.set_ylabel('Feature requests')
ax.legend()
plt.tight_layout()
save(fig, '08_feature_gaps_by_product.png')
plt.show()
```

For: Product (CPO)

What Customers Are Asking For and How Urgently

```
Saved: 08_feature_gaps_by_product.png
```

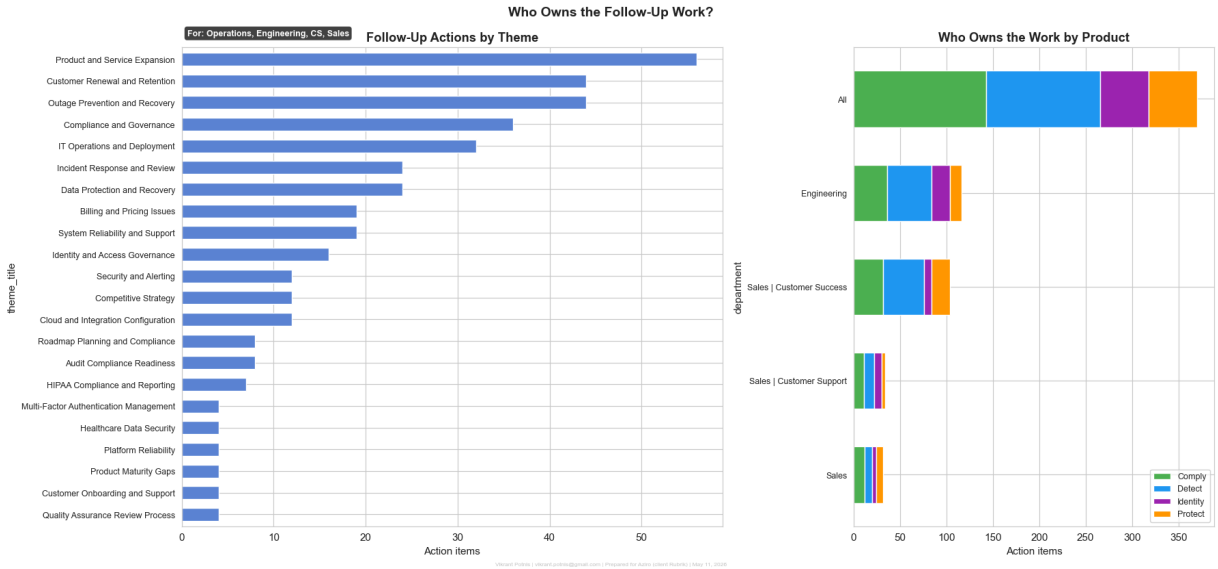


7.3 Who Owns the Follow-Up Work

For: Operations, Engineering, CS, Sales

Left: how many follow-up actions each theme generates. Right: which department owns those actions broken down by product — shows where bottlenecks sit.

In [15]:
```
# Query action item count per theme and per department x product
ai_theme = q('''
    SELECT theme_title, audience, count(*) AS action_items
    FROM meeting_analytics.action_items_by_theme
    GROUP BY theme_title, audience ORDER BY action_items DESC
''')
ai_dept = q('''
    SELECT abt.audience AS department, unnest(ms.products) AS product,
           count(*) AS action_items
    FROM meeting_analytics.action_items_by_theme abt
    JOIN meeting_analytics.meeting_summaries ms ON abt.meeting_id = ms.meeti
    WHERE array_length(ms.products, 1) > 0
    GROUP BY abt.audience, product
''')

# Aggregate and sort: themes by total action items, departments by total wor
theme_totals = ai_theme.groupby('theme_title')['action_items'].sum().sort_va
pivot_dp = ai_dept.pivot_table(index='department', columns='product',
                               values='action_items', fill_value=0).astype(
pivot_dp = pivot_dp.loc[pivot_dp.sum(axis=1).sort_values().index]

product_colors = {'Detect': '#2196F3', 'Comply': '#4CAF50',
                  'Protect': '#FF9800', 'Identity': '#9C27B0'}

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(18, max(8, len(theme_totals) *
                               gridspec_kw={'width_ratios': [3, 2]})
```

```
# Left: horizontal bar — action items by theme
theme_totals.plot.barh(ax=ax1, color='#5c85d6', width=0.6)
stakeholder(ax1, 'Operations, Engineering, CS, Sales')
ax1.set_title('Follow-Up Actions by Theme', fontweight='bold')
ax1.set_xlabel('Action items')
ax1.tick_params(axis='y', labelsize=9)

# Right: stacked horizontal bar — action items by department, coloured by pr
pivot_dp.plot.barh(stacked=True, ax=ax2,
                   color=[product_colors.get(c, '#9E9E9E') for c in pivot_dp
                   width=0.6)
ax2.set_title('Who Owns the Work by Product', fontweight='bold')
ax2.set_xlabel('Action items')
ax2.tick_params(axis='y', labelsize=9)
ax2.legend(loc='lower right', fontsize=9)

fig.suptitle('Who Owns the Follow-Up Work?', fontsize=14, fontweight='bold')
plt.tight_layout()
save(fig, '09_action_item_owners.png')
plt.show()
```

Saved: 09_action_item_owners.png